

Unity 2022.3 内置管线下完成一款 16×16 像素卡牌游戏的转型计划

执行摘要

对你这种“Unity 通才、原型快、但没有真正完结过像素游戏”的开发背景来说，最大的风险不是技术不会，而是在像素规则、卡牌可读性、导入规范、UI 体系和资源治理之间没有先定死约束，结果到了中后期才发现：美术风格不统一、卡面信息塞不下、相机与缩放让像素发虚、Atlas 与 Canvas 结构反而拖慢迭代。Unity 2022.3 LTS 在 Built-in Render Pipeline 下完全能做出稳定、清晰的像素卡牌游戏，但前提是你把项目当成“**有严格像素法则的 UI-heavy 2D 产品**”来设计，而不是普通 game jam 2D 项目。Unity 官方的 2D Pixel Perfect、Sprite Import、Sprite Atlas、2D Sorting、UGUI 性能与 Built-in Shader 文档，连同 Aseprite 与高质量像素教程，都指向同一个结论：**统一像素密度、统一缩放策略、统一 pivot/命名/导入预设**，要比“后期补像素味道”更关键。 ¹

本报告最重要的判断是：**不要把“16×16”理解成“整张卡只有 16×16 像素”**。对数字卡牌游戏而言，历史上扑克牌与现代游戏 UX 都反复强调“功能优先于装饰、识别优先于细节、降低认知负荷、利用玩家已有的视觉习惯”。如果整张卡严格做成 16×16，你几乎不可能同时满足卡名、费用、稀有度、关键词、状态反馈与交互可读性。更可行的方式是把 **16×16 作为原子模组**：用于图标、资源符号、状态、角标、头像块、桌面装饰与 UI 栅格；而实际卡面尺寸用 16 的整数倍，例如 48×64、64×80 或 64×96 作为首轮试制尺寸。这样既保持像素法则，又不牺牲卡牌游戏的识别效率。 ²

在 Unity 侧，最稳妥的起点是：**Built-in RP + Pixel Perfect Camera + 全项目统一 PPU + Point 采样 + 关闭 mipmaps + 初期关闭压缩 + 关闭 MSAA/后处理抗锯齿**。Unity 的 Pixel Perfect 文档明确要求同场景 Sprite 使用一致的 PPU，并建议 Point Filter、无压缩、像素单位 pivot；Pixel Perfect Camera 的 **Upscale Render Texture** 与 **Pixel Snapping** 正是为避免亚像素抖动、非整数缩放和旋转后像素失真而设计。与此同时，Built-in RP 下新建 Sprite 默认使用 **Sprites-Default**，不会受场景灯光影响；如果给 16×16 资产加上复杂光照或后处理 AA，边缘和颜色很容易被“软化”。 ³

在牌桌和 UI 架构上，不建议“全 Canvas”，也不建议“全 SpriteRenderer”。更推荐**混合式结构**：桌面上的牌实体、手牌排布、拖拽、战场插槽与世界空间特效用 **SpriteRenderer** 体系；HUD、菜单、牌库列表、设置、战斗日志、弹窗与大卡详情用 **Canvas**。原因很直接：Unity UI 的批重建成本高，单个元素变化会弄脏整张 Canvas，而 SpriteRenderer 在排序层、组装卡牌子层、世界空间动效与共享材质/Atlas 上更自然；反过来，Canvas 在分辨率适配、安全区、文本与交互组件上又明显更强。 ⁴

如果你只想拿走一个第一阶段方案，我建议是：**桌面优先、横屏优先、参考分辨率先从 426×240 试起、全项目 PPU=16、世界牌面先做 64×80、卡面文字压缩到 1-2 行以内、其余说明放 hover 放大卡或侧边详情**。这样既比 320×180 更适合卡牌可读性，又仍然是非常“像素风”的画布密度；后续如果确定风格更粗更简，再回退到 320×180。这个建议是基于 Unity Pixel Perfect 的“Reference Resolution”思路、Saint11 对“先选主画布再整倍数放大”的实践，以及 Canvas Scaler 的缩放机制综合得出的工程起点。 ⁵

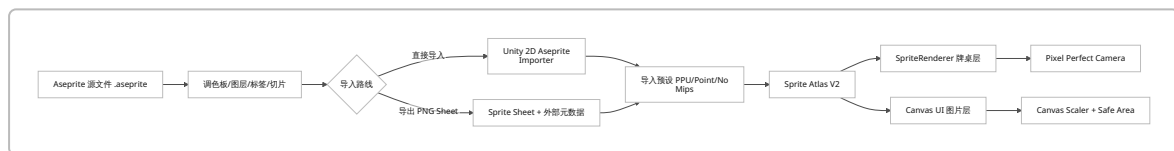
前提与建议架构

由于你没有指定目标平台、纵横比和输入方式，以下方案采用一个**保守但高完成度导向**的前提：先以 **PC 桌面横屏** 为主，鼠标操作为一等输入，游戏手柄为二等输入，触屏只做未来兼容；主支持 16:9，并兼顾 16:10 与更宽

显示器；如果未来要上移动端，再把安全区、触控 hit area、字体尺寸和压缩格式做第二轮适配。这个前提本身不是对“唯一正确方案”的断言，而是为了尽快收敛你的首个可完结版本。支持这一收敛策略的，是 Pixel Perfect/Canvas Scaler 一类系统本来就要求你先定义 Reference Resolution 与 UI 缩放基准，再扩展到更多屏幕。 6

从风格和生产性角度看，建议你把项目拆成三层：**低分辨率像素世界层、独立的卡牌交互层、高分辨率或逻辑分辨率的 UI 层**。低分辨率世界层负责桌布、背景、插槽、桌面装饰、简单粒子与世界空间牌实体；卡牌交互层负责手牌扇形、选中、拖拽、攻击指向、hover 高亮、阴影等；UI 层负责按钮、日志、设置、收藏册、构筑与大卡详情。这样可以把“像素纯度”和“可维护 UI”分开，避免所有东西都被同一套 Canvas 或同一张低分辨率 Render Target 绑定。Unity 的 Canvas、Sorting Group、Transparency Sort 与 Render Texture 体系都支持这种分层。 7

如果使用 Pixel Perfect Camera，Unity 已经帮你处理了“参考分辨率画布如何映射到实际屏幕”的大部分计算；如果你不使用 Pixel Perfect Camera，则可以把正交相机的首个估算公式写成： $\text{Orthographic Size} = \text{参考纵向像素} / (\text{PPU} \times 2)$ 。因此，如果你采用 $\text{PPU} = 16$ ，那么 426×240 的首轮参考分辨率对应 $\text{Orthographic Size} = 7.5$ ； 320×180 对应 5.625 。这不是 Unity 文档直接给出的固定值，而是由 Unity 对 PPU 与 Reference Resolution 的定义推导出来的工程公式，适合作为你不开 Pixel Perfect Camera 时的起点。 8



如果你希望尽快做出“能玩、能迭代、能完结”的垂直切片，我建议**不要一开始就追求复杂灯光、动态阴影或花哨的 3D 倾斜透视**。Built-in RP 默认使用的是旧的内置管线；若项目没有指定 Render Pipeline Asset，Unity 就会使用 Built-in RP。对于 16×16 像素与桌面卡牌玩法，Built-in RP 的优势不是视觉上限，而是结构简单、文档成熟、Sprite 默认材质直接可用。 9

关键陷阱与规避

像素游戏最常见、也最致命的坑，是**混分辨率、混 PPU、混缩放规则**。Saint11 对像素缩放与一致性的观点非常直接：混用不同“像素网格”会让整个视觉系统显得任意而不可信；而 Unity Pixel Perfect 也要求同一场景内 Sprite 使用统一 PPU。对你这种会 2D/3D、习惯快速混搭的开发者，这恰恰是最容易在早期原型里犯的错：角色 16×16 ，图标 12×12 ，卡框 58×81 ，结果最后只能靠模糊采样和后处理来“缝合”，像素感就被破坏了。规避方式是：**项目立项时只定义一套世界像素网格与若干整数倍模块，不允许自由尺寸漂移**。 10

第二个像素坑是把“像素风”误认为“只要锯齿和低分辨率就行”。真正影响成品质量的，不是画布够不够小，而是是否遵守轮廓、聚簇、色阶和抗锯齿规则。Saint11 对手工 AA 的建议是优先处理长 stair-step 与曲线转折，用过渡色减少锯齿；Pixel Parmesan 与 Saint11 都强调 banding/pillow shading 会把形体压扁、让作品发糊；Saint11 也提醒 dithering 是“作弊色阶/做材质”的方法，但用错会显得很噪。对于 16×16 这种极小画布，**任何多余的抖动、过多半色、过细的内轮廓都会迅速吃掉可读性**。所以你的默认规则应该是：先大块形体和高对比，再考虑少量 AA 与极少量抖动，而不是反过来。 11

卡牌游戏的核心坑则不是“像素不够像素”，而是**卡面和牌桌信息密度失控**。GDC 关于扑克牌 UX 的总结强调了 function over aesthetics、consistency、recognition 与 cognitive load；桌游界也长期强调图形设计会直接影响规则“是否更容易学习、记住和执行”；而游戏 UX 研究又反复指出，应尽可能降低认知负荷与错误 affordance。翻译成数字像素卡牌的落地原则就是：**玩家在 1 秒内必须看出卡牌类别、费用、战斗值、可用性和当前状态；任何需要停下来细看才能确认的核心信息，都是布局失败**。 12

第三个高频坑是**把所有桌面牌、HUD、日志、构筑列表和浮动说明都堆到一个大 Canvas 里**。Unity 官方明确说明：当一个 Canvas 上的单个元素变化时，整张 Canvas 都可能需要重新分析和重建 batch；大型列表、网格，以及像“卡牌战斗游戏里层层堆叠的卡”这样的重叠 UI，也会带来昂贵的 overdraw；嵌套 Layout Group 会在 dirty 流程中反复 `GetComponent`，非常伤性能。对卡牌游戏来说，这意味着“手牌 hover 一张、全 UI 抖一下 CPU”是完全可能发生的。规避方式不是“写更猛的机器”，而是**把动态与静态 UI 分 Canvas，热点区域尽量别用 Layout Group 过度自动排版，列表用池化，能用 SpriteRenderer 表示的牌桌对象尽量不要塞进 UGUI**。 ¹³

最后一个常见坑是**以为后期再统一 pivot、排序和命名就行**。实际上，Unity 的排序体系里，透明队列下 2D 渲染顺序会受到 Sorting Layer、Order in Layer、与相机距离、Sort Point、Sorting Group、材质/着色器等多重因素影响；一旦你前期没有明确 pivot 和组装约定，到了手牌扇形、拖拽抬层、大卡放大、世界空间 tooltip、牌堆动画阶段，会出现一连串看似随机的排序 bug。解决方式是：**卡类对象必须标准化 pivot；复合卡对象必须包在 Sorting Group 中；抬层/拖拽只改一个显式层级参数，不允许写很多临时补丁**。 ¹⁴

16×16 资产工作流

对这类项目，推荐你把 `.aseprite` 作为**唯一源文件**，而不是把导出的 PNG 视为源。Aseprite 官方文档明确说，`.aseprite` 文件会保留 color mode、layers、frames、palette、tags、slices 等完整信息；这意味着你可以把调色板、动画标签、切片、导出边界与注释都留在源文件里，再决定是让 Unity 直接导入，还是导出成 PNG Sheet 给其他流程。 ¹⁵

在图形方法上，16×16 的最佳实践是“**先轮廓、再色块、后细节**”。Saint11 的 cluster sketching 强调先用大色块建立形体，再逐步收敛；颜色上优先“用尽量少的颜色做尽量多的事情”；而手工 AA 只在最需要的阶梯边处理。我的建议是，你在首周只允许自己用**一套主调色板 + 一套中性色 UI 调色板**，并且强迫自己每种卡牌类别共享同一套高层视觉规则：费用角标永远同位置，资源色永远同方向，状态图标永远占一个 16×16 模块，卡图剪影优先于细节。这样才能让后续几十到几百张卡的生产仍然统一。 ¹⁶

一个非常实用的转型原则是：**只有“游戏原子”必须是 16×16，容器不必是 16×16**。所以你可以这样分解一张卡：左上费用 16×16，右上稀有度/阵营 16×16，中央立绘区 32×32 或 48×32，下方战斗值与关键词各占 16×16 模块；整张卡再由 3×4 或 4×5 个模组拼成。这样做的好处，是你复用了大量 16×16 图标，同时避免把文本和图像硬塞进一个无法阅读的小方块。这个方案不是官方规定，而是结合卡牌 UX 与像素模组化生产得出的工程建议。 ¹⁷

在 anti-aliasing、dithering 与线处理上，可以把团队规则写得非常明确：**默认不用引擎 AA，只允许手工 AA；默认不用大面积抖动，只在材质感与过渡需要时小范围使用；默认避免 banding 与 pillow shading；默认不做比 1 像素更细的“高频纹理”**。这些规则看起来死板，但正因为你有大量快节奏原型经验，更应该用规则抵消“凭感觉修一修”的冲动。 ¹⁸

Aseprite 的 `Tags` 和 `Slices` 对卡牌项目尤其有价值。Tag 可以把 hover、idle、deal、flip、burn、buff 等动画状态分段管理，支持 Forward / Reverse / Ping-pong；Slice 则可以给资产标记命名区域，并支持 9-slice / 9-patch 信息。对于卡牌游戏，这意味着你完全可以把“按钮框、对话框、卡框、悬浮提示底板”做成像素 9-slice 资源，而不是每个尺寸都手工重画。 ¹⁹

如果你想走“直接导入 `.aseprite` 到 Unity”的路线，Unity 的 2D Aseprite Importer 在 2022.3 可用，支持导入 `.ase` / `.aseprite`，并提供如 `Pivot Space`、`Import Mode`、`Sorting Group` 等特性；其中 `Pivot Space = Canvas` 对动画换帧尤其有用，因为它按源画布坐标计算 pivot，更适合帧间切换。需要注意的一个小坑是：**如果你后续在 Aseprite 里新增 Tag，导出的 Animator Controller 不会自动把新 clip 接进去**，这一点要在流程文档里写清楚。 ²⁰

Aseprite 直导与 PNG 导出的取舍

方案	适合场景	优点	代价	我的建议
直接导入 .aseprite	单人/小团队，想保留动画标签、切片、pivot 语义	源文件信息完整，改动回流快，Unity 能理解更多 Aseprite 语义	依赖 Unity 包；动画控制器和导入细节要管理	优先推荐
导出 PNG Sheet	多工具链协作、需要引擎无关导出	资产更“扁平”、CI 更直接、别的工具也能吃	源语义容易丢失，需要额外记录 slices/tags/pivots	适合后期自动化或外包交付

这个取舍表的依据，是 Aseprite 官方对源文件完整信息的描述，以及 Unity 2D Aseprite Importer 对 .aseprite 文件支持与导入特性的说明。²¹

Pivot 与锚点约定

资产类型	推荐 Pivot	原因
世界空间卡牌本体	Bottom-Center 或精确“接触桌面点”	扇形排布、抬起、落下、缩放更稳定
头像/图标/状态符号	Center	便于缩放、旋转、着色和数字叠加
角色/棋子/战场单位	脚底点 / 接地点	排序与地面接触感更自然
9-slice UI 框	Center	与 Canvas / 版式计算更直观
特效片段	按效果源点	爆点、拖尾、火花不应跟随资源重心漂移

Unity 明确支持在 Sprite Editor 中把 Pivot 设为 Custom，并使用 Pixels 作为 pivot 单位；Sprite 的 Sort Point 也可以使用 Center 或 Pivot。因此，你的 pivot 约定必须先于动画与交互写死，否则牌扇和拖拽系统会越来越难修。²²

Unity 导入与 Built-in RP 像素设置

Unity 2022.3 下，像素卡牌项目最应该一开始就锁定的是：相机、采样、纹理、Atlas、后处理。官方 Pixel Perfect 文档给出的“准备 Sprite”步骤已经很接近最佳实践：统一 PPU、Filter Mode 设为 Point、Compression 设为 None、pivot 使用像素坐标；还可以开启 Pixel Snapping 或 Upscale Render Texture 来避免亚像素移动和非整数缩放。对 16×16 项目，这套规则不是“建议看看”，而是“最好做成导入预设和检查脚本”。²³

在 Built-in RP 下，务必确认项目没有挂接 SRP 的 Render Pipeline Asset；否则 Unity 不会处于 Built-in。Sprite 层的默认材质是 Sprites-Default，不会受到场景光照影响；如果你故意改成 Default-Diffuse，Sprite 会开始响应灯光。对于 16×16 像素和桌面卡牌这种强调边界和色阶的项目，默认应把像素一致性放在“动态光照表现”之前，所以基础牌桌和卡牌本体更适合沿用 Sprites-Default 或自写极简 Unlit，而不是一上来就加 Diffuse/Standard。²⁴

抗锯齿方面，Unity 的质量设置里 MSAA 取值可以是 0/2/4/8；Post-processing 的 FXAA / SMAA / TAA 都是图像级抗锯齿，目标就是“软化边缘”。对写实 3D 这很合理，但对像素风卡牌，这正是你通常不想要的结果。尤其 TAA 文档明确提到它会在时间上累积帧，参数里也写了更稳定通常意味着更模糊。因此，像素项目的默认选项应是：MSAA=0，Post AA 关闭；只有在某些高分辨率非像素 UI 叠层里，才考虑局部例外。²⁵

推荐的首版导入设置

下表不是“Unity 唯一正确值”，而是一个适合 **16×16 像素卡牌项目首版** 的高稳定预设。它综合了 Unity Pixel Perfect、Sprite 导入、Texture 导入、Sprite Atlas 与 Built-in 渲染文档。²⁶

项目	推荐值	说明
Texture Type	Sprite (2D and UI)	卡牌、美术、UI 图都统一用 Sprite 导入
Sprite Mode	Single / Multiple	单图用 Single，图集/动画帧用 Multiple
Pixels Per Unit	16	让 16×16 成为 1 世界单位的原子模组
Mesh Type	Full Rect	小于 32×32 的 Sprite 本来也会走 Full Rect；矩形卡牌也更稳定
Filter Mode	Point (no filter)	像素边界清晰的底线
Compression	None	首版先保真，再做平台 override
Generate Mipmaps	Off	牌桌/UI 通常不需要，且 mipmaps 额外占约 33% 大小
Wrap Mode	Clamp	避免边缘平铺污染
Read/Write	Off	除非运行时要改像素，否则会双倍内存
sRGB	On	普通颜色贴图默认开；数据贴图再例外关闭
Alpha is Transparency	透明精灵建议 On	可减轻透明边缘过滤伪影
Generate Physics Shape	Off	卡牌/图标通常不需要物理形状

几个容易忽略的点值得单独说明。第一，Unity 明确写到：**小于 32×32 的 Sprite 即便选了 Tight，也会使用 Full Rect**。这意味着对于 16×16 图标、状态符号、资源块，你不用把时间浪费在 Tight Mesh 上。第二，Read/Write 会让 Unity 为脚本访问保留一份副本，从而**翻倍纹理内存**；第三，mipmaps 对只按原始或整数倍分辨率显示的 UI 纹理基本没有好处，却会增加大小。²⁷

Pixel Perfect Camera 的首版建议

项目	建议
Assets Pixels Per Unit	16，与全项目 Sprite PPU 完全一致
Reference Resolution	先从 426×240 开始；极端复古才退到 320×180
Upscale Render Texture	开，当你需要更稳定的整数像素外观时
Pixel Snapping	对移动世界牌、桌面单位建议开
Crop Frame	视目标屏幕决定；优先保持比例，不要硬拉伸

项目	建议
Snap Settings	Move X/Y/Z = 1 / PPU = 0.0625

这些做法都直接来自 Unity 2D Pixel Perfect 对 Asset PPU、Reference Resolution、Upscale Render Texture、Pixel Snapping 与 Snap Settings 的定义。²⁸

单独 Sprite 与 Atlas 的取舍

方案	优点	风险	什么时候用
单独 Sprite 文件	迭代直观，局部替换方便，导入独立	纹理多时 draw call 与状态切换更碎	原型期、小数量卡牌、频繁替换资产
按使用场景分 Sprite Atlas	减少 draw call，便于共享材质/批处理	只要场景里用到其中一张，Unity 会加载整张 Atlas；分得过大反而浪费	成品期；按“共同出现”的资源分 Atlas
超大总 Atlas	理论上最少切换	内存、加载和迭代都更差，像素项目经常得不偿失	通常不推荐

Unity 官方明确指出，Sprite Atlas 可以把多张纹理整合为一次纹理访问，从而减少 draw call；但同时也明确说明：**场景中只要一个 Sprite 激活，Unity 就会加载它所属 Atlas 及其包含的全部纹理**。因此，最好的做法不是“一个总 Atlas”，而是**按共同使用范围拆多个小 Atlas**。²⁹

Atlas 设置的实战建议

对卡牌与 UI atlas，建议：Allow Rotation = Off、Tight Packing = Off for cards/UI、Padding >= 4。原因很简单：Unity 官方写明了旋转打包会让 Canvas UI 纹理在场景中也发生旋转；Padding 的意义就是防止相邻 Sprite 发生像素重叠；默认值是 4。对于像素卡牌，卡框和角标大多是规则矩形，紧密 packing 的收益远小于调试成本。若后期做移动端压缩或 Variant Atlas，更要谨慎，因为 Unity 也提醒太低的 Variant 缩放可能带来可见伪影。³⁰

Built-in RP 下的着色器选择

方案	优点	代价	结论
Sprites-Default	默认可用，不受场景灯光影响，最稳	功能简单	牌桌与卡牌本体首选
Default-Diffuse	能响应灯光，成本比更复杂 shader 低	16×16 资产很容易因光照而失去色块清晰度	只用于少量需要受光对象
自定义 Unlit 像素 Shader	可做 palette swap、outline、hover tint、灰化	需自己维护共享材质和排序细节	中期推荐
Standard / 更复杂 Lit Shader	Built-in 支持，但更重	对像素卡牌通常收益很低	不建议作为基础方案

Unity 的 Sprite Renderer 文档、内置 shader 性能文档都支持这个结论：默认 Sprite 是 unlit，复杂 shader 一般按 Unlit → VertexLit → Diffuse → 更复杂的顺序增加成本。对 16×16 卡牌项目，先把 shader 当成功能层，而不是“美术救命层”。³¹

平面牌桌 UI、排序与布局

对“平面卡桌”场景，我最推荐的构成是：桌面牌实体走 **SpriteRenderer**，屏幕 UI 走 **Canvas**。这不是教条，而是因为 Unity 的两个系统各自擅长不同的事：SpriteRenderer 更适合牌在世界中的排序、扇形排布、拖拽、阴影、高亮和世界空间 tooltip；Canvas 更适合菜单、日志、数值面板、构筑列表、设置页与安全区适配。尤其是你要做手牌 hover 放大、牌堆抽牌、战场进出场、选中目标线时，SpriteRenderer 会比把所有牌当成 UI Image 更自然。 ³²

卡牌尺寸上，我建议在首轮原型里只测试两种：**48×64** 与 **64×80**。前者更接近“像素粒感强、信息非常克制”的风格，后者则对数字卡牌的状态可读性更友好。无论选哪种，都把卡面分成固定模组，而不是每张卡都随内容自由排版。这样做的真正收益不在“更好看”，而在于你能快速复用角标、资源框、职业徽记、稀有度条、状态位等组件。这个尺寸建议是工程综合建议，不是 Unity 文档中的硬指标。它的依据在于卡牌 UX 对识别/一致性的要求，以及像素项目对模组化生产的需要。 ³³

UGUI 侧要牢记一条 Unity 官方反复强调的原则：**静态和动态 UI 分 Canvas**。如果整个游戏 UI 都在一张大 Canvas 上，那么 hover 高亮一个按钮、更新一条日志、改变一张手牌都会让整张 Canvas 重建；而 Child Canvas 可以把内容隔离开。对卡牌游戏尤其建议拆成：全局 HUD Canvas、日志/弹窗 Canvas、手牌详情 Canvas、过场/转场 Canvas。那些频繁更新的热点区域，尽量不要和几乎不动的装饰性 UI 在同一张 Canvas 里。 ³⁴

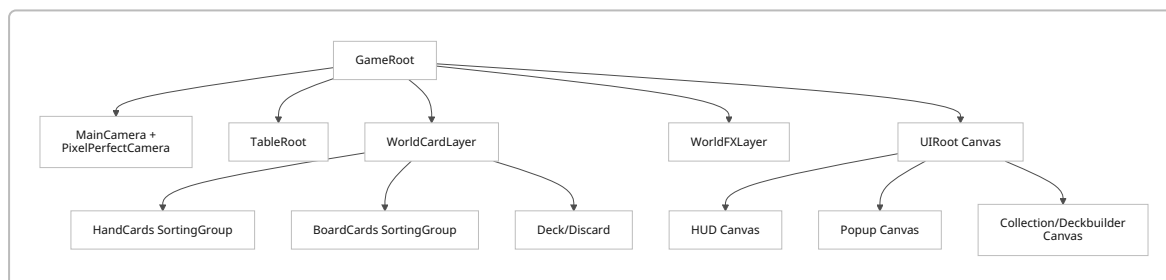
如果你做 Screen Space UI，请用 `Canvas Scaler -> Scale With Screen Size`，并明确一个 Reference Resolution。Unity 文档解释得很清楚：这种模式会以参考分辨率为布局基准，再按屏幕尺寸放大或缩小；当屏幕比例不同，还可以通过 `Screen Match Mode` 决定偏向宽度还是高度。对卡牌游戏，我通常建议主 HUD 采用这个模式，而不是 Constant Pixel Size，因为后者在不同显示器上更容易出现过小/过大 UI。 ³⁵

如果未来考虑移动端、Steam Deck 或带安全区的设备，`Screen.safeArea` 应该在 UI 容器层预留，而不是等适配时再补。Apple HIG 也强调了 safe area 的重要性；同时，如果有触屏输入，Apple 建议按钮至少约 `44×44 pt`，Material 建议至少 `48×48 dp` 的触达区域。即便你现在以 PC 为主，把这些约束做成可切换模式也能减少后期返工。 ³⁶

SpriteRenderer 与 Canvas 的对比

维度	SpriteRenderer	Canvas UI
世界中的牌排序	强，天然适合 Sorting Layer / Order / Sorting Group	弱，更多依赖层级与 Canvas 排序
自动布局与分辨率适配	需手写	强，Canvas Scaler/Anchor 很成熟
文本与按钮系统	需额外实现	强
hover/拖拽/牌桌动效	很自然	能做，但容易变成重 UGUI 工程
大量动态对象性能	常更稳定，尤其共享材质/Atlas 时	如果热区全挤一张 Canvas，重建成本高
安全区与设备 UI 适配	需自己处理	原生更方便

这个表的核心依据，是 Unity 对 Canvas 重建成本、Hierarchy 绘制顺序、Sorting Group 与 2D Sorting 的说明。 37



一个可执行的排序约定

你不必追求“无限灵活”，反而应追求**显式排序规则**。一个够用的首版可以是：

- BG_Table
- Board_Slots
- Cards_Back
- Cards_Front
- Cards_Drag
- World_FX
- World_Tooltip
- UI_Base
- UI_Popup
- UI_Debug

再配合以下规则：复合卡对象全部带 `SortingGroup`；一张卡的子元素不允许直接改独立 `Sorting Layer`；抬起/拖拽只提升根对象层级；Screen Space UGUI 通过 sibling order 控制覆盖关系。这样你的排序问题会从“玄学”变成“配置问题”。Unity 对 2D Sorting 的优先级链条与 Canvas 的 Hierarchy 绘制顺序，都支持这种做法。

38

命名、版本治理、性能与 QA

Unity 官方关于项目组织的建议很值得像素项目直接照抄：**文档化命名规则**；**不要在文件/文件夹名里用空格**；**把自家资产和第三方资产分开**；**尽量不要在项目根级乱开文件夹**；**移动资源时要在 Unity Editor 里移动**，以保留 `.meta` 与历史。这是像素项目尤其重要的一点，因为你的导入设置、切片、pivot、Atlas 关系都高度依赖 `.meta`。如果你在系统外面乱移动资源，最先坏掉的往往不是代码，而是美术导入一致性。 39

在版本治理上，必须把 `.meta` 文件纳入版本控制。Unity 明确说明 `.meta` 保存了导入设置等信息；如果不提交它们，团队成员拿到同一张 PNG，实际导入结果可能完全不同。对于你这种将来很可能继续 solo 或小团队推进的人，这件事同样关键，因为它直接决定“半年后回来看工程还能不能无损继续”。此外，组织文章也提到，空文件夹在 Git/Perforce 下容易出问题；最好只创建需要的目录，必要时才用 `.keep`。 39

推荐的命名规则

我建议用“用途前缀 + 语义主体 + 状态/变体”的方式，而不是美术随手命名：

- crd_frame_common_a
- crd_icon_fire_16

- crd_cost_gem_blue
- tbl_slot_player_front
- ui_btn_confirm_idle
- ui_btn_confirm_hover
- fx_hit_spark_01
- pal_core_village_day
- ase_crd_frame_common_src

这里的关键不是前缀长得多漂亮，而是一眼能看出用途与加载范围。Unity 自己也强调，文件目录和命名应该尽可能显式表达资产“意图”，这样工程内其他人才能判断它在何时被加载、如何与别的资产打包。对卡牌游戏而言，这一点尤其适合 Atlas 和 Addressables 规划。 ⁴⁰

示例目录结构

下表是一个适合像素卡牌项目、同时兼顾“按用途”和“按资产类型”的混合结构。它遵循了 Unity“自家与第三方分离、统一模板化、记录命名标准”的建议。 ⁴¹

路径	内容	备注
Assets/Game/Art/Source/Aseprite	.aseprite 源文件	唯一美术真源
Assets/Game/Art/Palettes	调色板、参考图	调色板命名固定
Assets/Game/Art/Export/Sprites	导出的 PNG/Sheet	若不用直导则在此
Assets/Game/Art/Atlases	Sprite Atlas 资源	按共同使用场景拆分
Assets/Game/Materials	像素材质、hover/灰化材质	保持共享材质数量少
Assets/Game/Prefabs/Cards	卡牌表现 Prefab	世界空间牌对象
Assets/Game/Prefab/UI	HUD、面板、弹窗预制体	Screen Space 为主
Assets/Game/Data/Cards	ScriptableObject / JSON	卡牌规则与数值
Assets/Game/Scenes	Boot Table Deckbuilder DevSandbox	Sandbox 单独隔离
Assets/Game/Scripts/Runtime	运行时代码	
Assets/Game/Scripts/Editor	导入器、检查器、工具	AssetPostprocessor 放这里
Assets/ThirdParty	第三方插件与商店资源	与自家资源隔离

导入规范方面，最值得做成制度的是 **Preset + AssetPostprocessor** 双保险。Unity 的 Preset 能给 importer 指定默认设置，还支持过滤；`AssetPostprocessor.OnPreprocessTexture()` 可以在导入前统一改纹理压缩、PPU、Filter 等默认值。这意味着你完全可以让所有 `Assets/Game/Art/**` 下的图片自动套用 `Point + No Mips + PPU=16`，而不靠人肉点 Inspector。 ⁴²

16×16 资产的性能、内存与包体策略

16×16 像素资产的好消息是：原始数据本来就很小。Unity 的 `TextureFormat.RGBA32` 是每通道 8 bit、共 4 byte/像素，因此一个不带 mipmaps 的 16×16 RGBA32 纹理，原始像素数据只有 $16 \times 16 \times 4 = 1024$ bytes；64×80 的卡面也不过约 20 KB 级别的原始数据量。对这种体量，很多项目在桌面端最好的策略反而是“先不压缩，先保证边缘和色块不坏”，等平台明确后再做 per-platform override。⁴³

真正更值得警惕的，是**隐性开销**而不是基础像素量：

一是 mipmaps 会额外增加约 33% 的磁盘与内存占用；二是 `Read/Write` 会翻倍纹理内存；三是 Atlas 只要命中其中一张 Sprite，整张 Atlas 就会被加载；四是压缩格式在非常小的高对比像素图上，主观伪影往往比在大纹理上更明显。这四项里，前三项都有明确的 Unity 文档支持；第四项是工程推论，因此更适合作为“压缩前必须做肉眼对比测试”的团队规则。⁴⁴

如果未来要做平台 override，可用一个保守策略：桌面先 `None`，确认稳定后再测试 `BC7/DXT5`；现代 iOS / Android 以 `ASTC 4x4` 作为首个候选；需要兼容更老 Android 时测试 `ETC2`，并特别观察 alpha 边缘。如果支持 very old iOS，才考虑 `PVRTC`。Unity 官方对桌面、iOS/tvOS、Android 的推荐压缩格式都写得很清楚，但对像素卡牌这种高对比小图，我会把“视觉验收”置于“理论压缩收益”之前。⁴⁵

在运行时性能上，不要默认迷信 Dynamic Batching。Unity 明确说明，Dynamic Batching 最初是为旧低端设备设计的；在现代硬件上，它在 CPU 上做的转换工作可能反而比 draw call 更贵，因此应该**开关对比、用 Profiler 说话**。真正更稳定的收益，通常来自：共享材质、合理 Atlas 切分、减少 UI 重建、避免巨量重叠 Image、控制 Animator 对 UI 的持续 dirty。⁴⁶

常见视觉 bug 与修复对照

症状	常见原因	优先修复
像素发糊	Bilinear/Trilinear、mipmaps、压缩、后处理 AA、非整数缩放	Point、关 mips、关压缩、关 MSAA/FXAA/TAA、上 Pixel Perfect
移动时抖动/闪烁	亚像素移动、相机非整数位移	开 Pixel Snapping, Snap=1/PPU，动画改整数像素步进
Atlas 邻边串色	Padding 太小、Clamp/Atlas 配置不当、旋转打包	Padding ≥ 4 ，卡牌/UI 关 Rotation，检查 Pack Preview
透明边缘黑边/白边	alpha 边缘处理不当	透明 Sprite 开 <code>Alpha is Transparency</code> ，先用无压缩验证
手牌扇形偏心	pivot 不统一	统一 bottom-center 或接触点 pivot
卡牌子元素前后关系乱	没有 Sorting Group，或排序逻辑有多个入口	全部复合卡加 Sorting Group，只允许根对象改层
UI CPU 峰值	单巨型 Canvas、Layout Group 套娃、频繁后停对象	动静分 Canvas，少用 Layout Group，按官方方式池化
刘海/圆角裁切 UI	没处理 Safe Area	UI 根节点包一层 Safe Area 容器

这个排查表的依据主要来自 Unity 对 Pixel Perfect、Sprite Import、Sprite Atlas、Canvas 性能、2D Sorting 与 Safe Area 的文档说明。 47

测试与 QA 清单

首轮 QA 不要只在一个分辨率看效果。至少测试：1280×720、1366×768、1600×900、1920×1080、2560×1440，以及一种超宽比例。测试重点不是“能显示”，而是：

- 是否始终保持整数缩放或至少没有明显软化。
- 手牌 hover 放大后，信息层级是否仍稳定。
- 手牌、战场卡、tooltip、伤害/增益数值的覆盖顺序是否永远一致。
- 卡牌拖拽时，阴影、高亮、目标线是否都从属于同一排序逻辑。
- 打开/关闭日志、收藏册、设置面板时是否出现 Canvas 重建尖峰。
- 关闭/开启动态批处理后帧时间谁更好。
- 如果预留移动端方案，Safe Area 与触控 hit area 是否合格。 48

初始冲刺清单

下面这份清单尽量做到了“**本周就能开工**”，同时覆盖了 art、import、prototype 三条线：

- 确定项目基线：Built-in RP、PPU=16、Reference Resolution=426×240、MSAA=0。 49
- 建立目录模板与命名约定；开启可见 .meta；提交到版本库。 39
- 选择一套主调色板与一套 UI 中性色板；写成项目说明。 50
- 在 Aseprite 里先做 1 套卡框、3 套资源图标、1 套状态图标、1 套桌面插槽。 51
- 建一个导入 Preset 或 AssetPostprocessor，把 Point / No Mips / Compression None / PPU 16 自动化。 52
- 建两个 Atlas：Table_Core 与 UI_Core；卡牌/UI atlas 关 Rotation。 53
- 做一个最小牌桌：牌堆、弃牌堆、两手牌、两战场槽位、hover 大卡。 54
- 手牌与战场牌都用统一 pivot + SortingGroup。 55
- HUD、日志、弹窗拆成至少 3 张 Canvas。 56
- 只做 8-12 张测试卡，但覆盖 3 种视觉类别：单位、法术、装备/遗物。
- 每张测试卡都验证小图、hover 放大图、详情面板这三种展示。
- 第一周结束前用 Frame Debugger / Profiler 看一次真实批次和 UI 重建，别等容量上来才看。 57

工具、参考资料与开放问题

优先推荐的工具与文档

Unity 官方优先级最高。建议你以下内容当成“项目规范源文件”而不是随手查阅的知识点：

2D Pixel Perfect、Sprite (2D and UI) Import Settings、Sprite Atlas / Workflow / V2、2D Sorting、Sorting Group、Canvas、Canvas Scaler、Screen.safeArea、Asset Workflow、Preset Manager、AssetPostprocessor、Built-in Render Pipeline 与 Sprite Renderer。这些文档几乎覆盖了本报告里所有工程约束。 58

像素创作工具首推 Aseprite。原因不是“它最潮”，而是它对像素工作流真正有用：源文件能保留 layers / frames / palette / tags / slices，Tag 和 Slice 又恰好能服务于你的卡牌动效与 9-slice 需求。若你想让 Unity 更直接理解这些语义，2022.3 可用 2D Aseprite Importer。 59

像素方法论参考首推 Saint11 与 Lospec。Saint11 的 AA、banding、shading、consistency、scaling 与 animation 文章，非常适合已经会做游戏、但还没形成像素“规矩”的开发者；Lospec 的 Palette List 与教程库则适合做调色、查风格、找学习路线。 60

UI 参考资料方面，如果你需要大量看现成游戏界面，Edd Coates 的 Game UI Database 是业内知名参考工具；如果你需要更系统的游戏 UX 思路，GDC 上关于扑克牌演化、认知负荷、affordance 与界面设计的内容很值得补。它们不会替你选尺寸，但会帮你做对“信息优先级”的判断。 61

开放问题与局限

这份方案在工程上已经足够开工，但仍有几个影响最终参数的未定项：

- **目标平台未定**：它会影响你是否接受纹理压缩、safe area 优先级、触控 hit area、字体系统和输入 affordance。
- **卡牌文本密度未定**：如果你走《杀戮尖塔》式短关键词，卡面可更小；如果是 CCG 式长描述，必须尽早引入 hover 大卡/详情面板。
- **纵横比与镜头风格未定**：若你想做更明显的透视倾斜和 2.5D 桌面，Sorting、pivot 和 shader 设计都会更复杂。
- **是否全像素文本未定**：若后续要多语言、本地化和大卡数据库，建议尽快决定“卡框像素化，但文本系统化”还是“整张卡 baked sprite”。

在这些信息未定之前，报告中关于 426×240、PPU=16、64×80 卡面、桌面优先 等数值，应理解为高可执行的起始基线，而不是唯一答案。

1 3 5 6 8 22 23 26 28 47 55 <https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal%4012.1/manual/2d-pixelperfect.html>

<https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal%4012.1/manual/2d-pixelperfect.html>

2 12 17 33 <https://gdcvault.com/play/1027289/UX-Summit-History-Shaping-Design>

<https://gdcvault.com/play/1027289/UX-Summit-History-Shaping-Design>

4 13 34 37 48 56 57 <https://unity.com/how-to/unity-ui-optimization-tips>

<https://unity.com/how-to/unity-ui-optimization-tips>

7 <https://docs.unity3d.com/2022.3/Documentation/Manual/UIColorCanvas.html>

<https://docs.unity3d.com/2022.3/Documentation/Manual/UIColorCanvas.html>

9 24 49 <https://docs.unity3d.com/2022.3/Documentation/Manual/srp-setting-render-pipeline-asset.html>

<https://docs.unity3d.com/2022.3/Documentation/Manual/srp-setting-render-pipeline-asset.html>

10 <https://saint11.art/blog/scaling/>

<https://saint11.art/blog/scaling/>

11 18 60 https://saint11.art/pixel_art_articles/article5/

https://saint11.art/pixel_art_articles/article5/

14 38 54 <https://docs.unity3d.com/cn/2022.3/Manual/2DSorting.html>

<https://docs.unity3d.com/cn/2022.3/Manual/2DSorting.html>

15 21 51 59 <https://aseprite.com/docs/files/>

<https://aseprite.com/docs/files/>

16 https://saint11.art/pixel_art_articles/article6/
https://saint11.art/pixel_art_articles/article6/

19 <https://aseprite.com/docs/tags/>
<https://aseprite.com/docs/tags/>

20 <https://docs.unity3d.com/2022.3/Documentation/Manual/com.unity.2d.aseprite.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/com.unity.2d.aseprite.html>

25 <https://docs.unity3d.com/2022.3/Documentation/ScriptReference/QualitySettings-antiAliasing.html>
<https://docs.unity3d.com/2022.3/Documentation/ScriptReference/QualitySettings-antiAliasing.html>

27 <https://docs.unity3d.com/2022.3/Documentation/Manual/texture-type-sprite.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/texture-type-sprite.html>

29 <https://docs.unity3d.com/2022.3/Documentation/Manual/sprite-atlas.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/sprite-atlas.html>

30 <https://docs.unity3d.com/2022.3/Documentation/Manual/class-SpriteAtlas.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/class-SpriteAtlas.html>

31 32 <https://docs.unity3d.com/2022.3/Documentation/Manual/class-SpriteRenderer.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/class-SpriteRenderer.html>

35 <https://docs.unity3d.com/2022.3/Documentation/Manual/script-CanvasScaler.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/script-CanvasScaler.html>

36 <https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Screen-safeArea.html>
<https://docs.unity3d.com/2022.3/Documentation/ScriptReference/Screen-safeArea.html>

39 41 <https://unity.com/how-to/organizing-your-project>
<https://unity.com/how-to/organizing-your-project>

40 <https://unity.com/blog/engine-platform/why-folder-structures-matter>
<https://unity.com/blog/engine-platform/why-folder-structures-matter>

42 52 <https://docs.unity3d.com/2022.3/Documentation/Manual/class-PresetManager.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/class-PresetManager.html>

43 <https://docs.unity3d.com/2022.3/Documentation/ScriptReference/TextureFormat.RGBA32.html>
<https://docs.unity3d.com/2022.3/Documentation/ScriptReference/TextureFormat.RGBA32.html>

44 <https://docs.unity3d.com/6000.0/Documentation/Manual/texture-mipmaps-introduction.html>
<https://docs.unity3d.com/6000.0/Documentation/Manual/texture-mipmaps-introduction.html>

45 <https://docs.unity3d.com/2022.3/Documentation/Manual/class-TextureImporterOverride.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/class-TextureImporterOverride.html>

46 <https://docs.unity3d.com/2022.3/Documentation/Manual/dynamic-batching.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/dynamic-batching.html>

50 <https://lospec.com/palette-list>
<https://lospec.com/palette-list>

53 <https://docs.unity3d.com/2022.3/Documentation/Manual/SpriteAtlasWorkflow.html>
<https://docs.unity3d.com/2022.3/Documentation/Manual/SpriteAtlasWorkflow.html>

58 <https://docs.unity3d.com/cn/2022.3/Manual/com.unity.2d.pixel-perfect.html>
<https://docs.unity3d.com/cn/2022.3/Manual/com.unity.2d.pixel-perfect.html>

61 <https://eddcoatesdesign.com/>
<https://eddcoatesdesign.com/>